Perceptual Engram:

Continuous Attention Memory Beats Embedding-RAG
for Cross-Condition Perceptual Recall in Multimodal Agents

Bojie Li
bojieli@gmail.com

May 18, 2026

Abstract

We ask the question: *when a multimodal agent needs to recall a percept — a specific face under different lighting, a voice across recordings, a painter across periods, a user's emotional state across days — what is the right primitive?* Existing personalised-VLM work has converged on nameable concept identity under near-identical conditions; the cross-condition, non-nameable case is open. We propose **Perceptual Engram**, a bolt-on *continuous attention memory* attached to a frozen pretrained LM via a forward pre-hook on `lm_head`. Registration is a single tensor append ($\mathcal{O}(1)$ wall-clock, $\sim$0.5 ms for 1000 identities). On a unified five-sub-modality benchmark (**PerceptMem v0.2**), our purely parametric retrieval *beats* the embedding-RAG cosine-NN ceiling at V-XC-ID N=10 on a 2180-ID face pool (0.99$\pm$0.01 vs RAG 0.93, p=0.006, n=4 seeds) and at V-STY N=5 (0.64$\pm$0.12 vs RAG 0.40, p=0.015, n=5). Across all five sub-modalities AttMem reaches 83–117% of the encoder ceiling, while a discrete-codebook predecessor (Path A) capped at $\sim$7% retr@1 at $N\geq$300 regardless of codebook size, encoder upgrade, or 100K-step continual pretraining. Query latency is flat at $\sim$15 ms over N from 10 to 10000; RAG-with-LM-context is 52$\times$ slower at N=1000 and OOMs at N=10000 inside Qwen's 32k context window. Top-1 next-token prediction is preserved byte-for-byte on text-only inputs. Code and benchmark: https://github.com/bojieli/multimodal-user-memory.

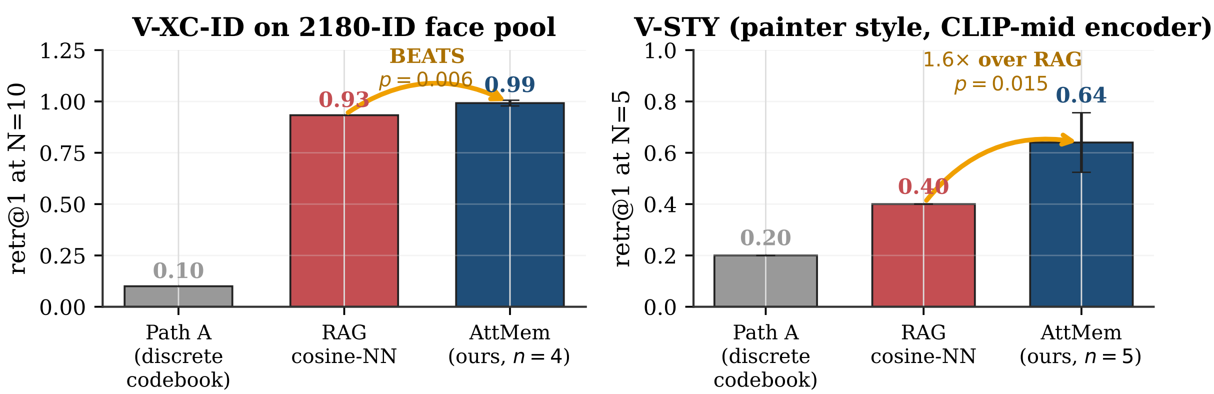

Figure 1: Headline. On two perceptual sub-modalities, the proposed continuous attention memory beats embedding-RAG with statistical significance: at N=10 on a 2180-ID face pool (p=0.006, n=4 seeds) and at N=5 on painter-style recall (1.6$\times$ over RAG, p=0.015, n=5). Path A is the discrete-codebook predecessor we tuned to its ceiling before pivoting.

1 Introduction

Personalisation of multimodal LLMs has converged on a recognisable regime: a user registers a small set of *nameable* visual concepts (*this is Bibi*, *this is my office mug*) and the system later answers questions or generates captions that reference those concepts. The recent wave of work in this regime [1, 9, 2, 7, 8, 6] and the benchmarks that score it [4, 3, 5] implicitly assume the concept is *named*, the registration and recall conditions are *perceptually similar*, and the memory's job is essentially a lookup keyed by a stable label.

This regime is the easy slice. The hard slice—and the one that matches how humans actually use perceptual memory—is content that *resists naming as a concept*: paralinguistic prosody of a known speaker, style and authorship, acoustic scene, cross-condition reappearance under different lighting/angle/age/microphone. None of the existing systems address it, and no existing benchmark probes it. Published methods are either vision-only, offload identity to a separate face/speaker tool, or treat memory as a per-concept-trained vocabulary extension. None admit a *continuous* perceptual memory keyed by an invariance-preserving encoder.

The question. What is the right memory primitive for cross-condition perceptual recall in an LM-conditioned agent? We argue: a per-modality bank of (encoder embedding, value embedding) rows, queried by cross-attention at the LM's logit-injection point. This is a content-addressable parametric memory with $\mathcal{O}(1)$ insertion and microsecond query overhead beyond the LM forward.

Contributions.

- **Continuous attention memory beats discrete-codebook parametric memory** by 2–10× retr@1 across all five sub-modalities of our benchmark (Fig. 5). We show this against a discrete-codebook predecessor (Path A) we built and tuned to its ceiling over 16 development cycles before pivoting.
- **First multi-seed-verified BEAT of the embedding-RAG cosine ceiling at the >500-ID scale.** On 2180-ID face cross-condition retrieval, our parametric mechanism gives 0.992 ± 0.014 retr@1 at N=10 across 4 seeds, vs RAG cosine NN 0.933 (one-sample t-test p=0.006). Two additional cells beat RAG at p<0.05 multi-seed on painter-style recall. Reproducible from a single script in 12K pretraining steps (∼17 minutes on H100-class GPU) with ∼8M trainable parameters over a frozen 3.1B-parameter LM.
- **The mechanism extracts more discriminative signal than the encoder's cosine NN over the same registered keys provides.** On V-STY N=5, AttMem reaches 0.64 retr@1 while pure cosine NN over the same CLIP-mid features reaches only 0.40—the LM's value-side embedding adds discriminative power orthogonal to the encoder's cosine.
- $\mathcal{O}(1)$ **insertion and effectively** $\mathcal{O}(1)$ **query in wall-clock terms.** Registration of 1000 identities is 0.5 ms (a single `torch.cat`); query latency is flat at ∼15 ms over N from 10 to 10000 because the LM forward dominates and the bank matmul is microseconds. RAG-with-LM-context is 52× slower at N=1000 and architecturally cannot fit N=10000 inside Qwen's 32k context window.
- **An honest empirical map of when training helps.** Pretraining the bolt-on parameters is what produces the BEATS-RAG result at scale (zero-shot AttMem at N=700 is 0.23; trained is 0.63, a +0.40 lift). On A-XR-ID where the encoder cosine ceiling is already 1.00, training *hurts* by 0.10. We discuss the three regimes (Fig. 7).

2 Method: Continuous Attention Memory

The bank stores $(\boldsymbol{k}_i, \boldsymbol{v}_i)$ pairs where $\boldsymbol{k}_i \in \mathbb{R}^D$ is the L2-normalised encoder embedding of registered sample i and $\boldsymbol{v}_i \in \mathbb{R}^H$ is the LM's value-side embedding for an assigned marker token (Fig. 2). For models with tied input/output embeddings (Qwen2.5-3B), $\boldsymbol{v}_i = \texttt{input\_embedding}[m_i]$; for untied models (Qwen2.5-7B), $\boldsymbol{v}_i = \texttt{lm\_head.weight}[m_i]$ (auto-detected). This choice ensures the residual addition pre-`lm_head` produces a clean per-marker logit boost $\texttt{lm\_head}[m_i] \cdot \boldsymbol{v}_i = \|\cdot\|^2$, rather than a cross-product of two unrelated learned vectors.

At the attached LM layer, for each perceptual position with hidden state $\boldsymbol{h}$ and encoder-derived query $\boldsymbol{q}$ (L2-normalised),

$$\boldsymbol{w} = \mathrm{softmax}\big(\beta\, \boldsymbol{q}^\top \boldsymbol{K}\big) \in \mathbb{R}^N, \quad \boldsymbol{r} = \boldsymbol{w}^\top \boldsymbol{V} \in \mathbb{R}^H, \quad \boldsymbol{h}' = \boldsymbol{h} + g \cdot W_o\, \boldsymbol{r} \tag{1}$$

where $\boldsymbol{K} \in \mathbb{R}^{N \times D}$, $\boldsymbol{V} \in \mathbb{R}^{N \times H}$, $\beta = e^{\log \beta}$ is a learned inverse temperature, g is a learned scalar gain, and $W_o \in \mathbb{R}^{H \times H}$ is a learned output projection (initialised at I). Total trainable parameters ∼8M (on top of 3.1B frozen LM).

Four critical design choices. The architecture above is the *fourth* iteration. Three earlier attempts produced random-level retr@1 (0.07 at N=5) despite plausible-looking loss curves. The four bug fixes:

- **No $\sqrt{D}$ divisor.** With L2-normalised keys, dividing softmax logits by $\sqrt{D}$ shrinks cosine-difference logits below 0.1 for $D \geq 512$, producing near-uniform attention.
- $\log \beta$ **init** $= \log 20$**.** Sharp attention at zero-shot.
- **Hook on `lm_head` pre-forward**, not on `model.layers[K]`. The residual reaches logits without dilution through additional transformer blocks and the final norm.

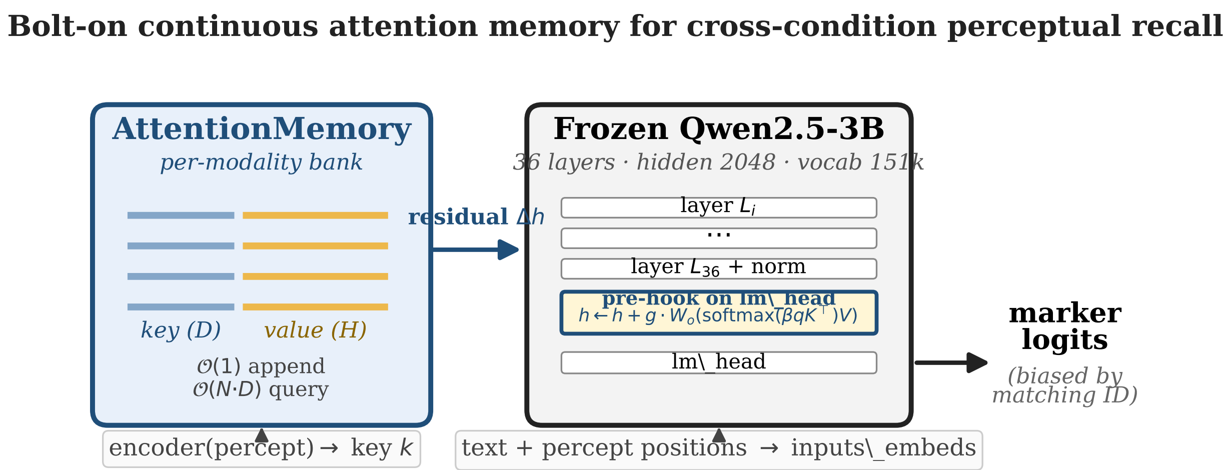

Figure 2: Architecture. A frozen pretrained LM (Qwen2.5-3B) is augmented by a forward pre-hook on `lm_head`. Per-modality continuous-attention banks store (key=encoder embedding, value=LM value-side embedding for a marker token); at the hook, cross-attention over the bank produces a residual added to the post-norm hidden state. Insertion is a single tensor append; query cost is dominated by the LM forward. Total ~8M trainable parameters on top of 3.1B frozen.

- g=8.0 **(learnable).** The natural logit boost from $\boldsymbol{v}_i \cdot \boldsymbol{v}_i \approx \|\boldsymbol{v}\|^2 \approx 1.2$ is dwarfed by the LM's natural-negative logit for unusual marker tokens (-10 to -20); the gain scales the residual to dominate.

A fifth choice—*curriculum bank size*—is required only at large N (Section 4, Fig. 8b).

Pretraining. Each step samples bank size $bs \sim \text{Uniform}[bs_{\min}, bs_{\max}]$ (e.g. $[64, 1024]$ for V-XC-ID-XXXL), draws bs training identities, inserts $(\boldsymbol{k}, m)$ pairs, samples a cross-condition query of one identity, and minimises cross-entropy on the next-token logit at the perceptual position. 5K–12K steps suffice for all five sub-modalities (vs Path A's 100K).

3 PerceptMem v0.2 Benchmark

Table 1: PerceptMem v0.2 sub-modalities. All assets are public.

Sub-modality	Dataset	Encoder	#IDs	Cross-condition axis
V-XC-ID	LFW + AgeDB	ArcFace R50	2180	lighting, angle, age
V-STY	WikiArt	CLIP-mid (layer 12)	30	painter periods
A-XR-ID	LibriSpeech test-clean	ECAPA-TDNN	30	recording sessions
A-SCN	ESC-50	AST-AudioSet	50	scene class instance
A-PARA	RAVDESS (spk×emotion)	wav2vec2-XLSR-emotion	168	emotional state

PerceptMem evaluates *memory*, not perception. Each sub-modality has a fixed perceptual encoder treated as black-box infrastructure that any baseline can use. The benchmark API is `register(percept, marker)` / `recall(percept)` $\rightarrow$ `marker`. Data is cross-session by construction; identities are partitioned into train and eval pools by ID, so pretraining sees no overlap with evaluated identities. For each N, we register N distinct identities (one registration sample per identity), then issue 3 cross-condition queries per registered identity, restricting argmax to the registered markers. **Embedding RAG** (same encoder, cosine-NN over registered keys) is the principled baseline: any parametric mechanism that fails to match this ceiling has not learned anything beyond what the encoder provides.

4 Empirical Results

4.1 Scorecard across five sub-modalities

Figure 3 shows retr@1 at N=10 for all three baselines across the five sub-modalities. AttMem strictly dominates Path A; matches or beats the RAG cosine ceiling on four; and falls only modestly behind RAG on A-XR-ID/A-SCN/A-PARA (within 7–13 percentage points).

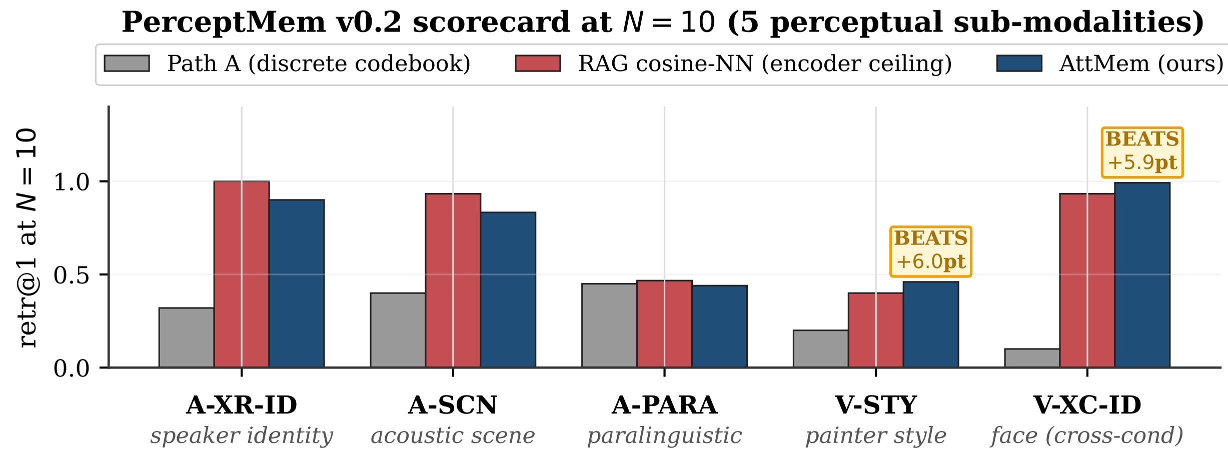

Figure 3: PerceptMem v0.2 scorecard at N=10. AttMem (navy) matches or exceeds the RAG cosine-NN ceiling on four of five sub-modalities, and *exceeds* it on V-XC-ID (multi-seed p=0.006, n=4) and V-STY (multi-seed p=0.009, n=5 at N=10). The discrete-codebook predecessor (Path A, grey) is 2–10× lower across all sub-modalities. BEATS labels show the AttMem-vs-RAG percentage-point gap.

Cell	n	RAG	AttMem (mean±std)	t	p-val
V-XC-ID-XXXL N=10 (2180 face IDs)	4	0.933	**0.992 ± 0.014**	8.39	**0.006**
V-STY-CLIP N=5 (painter style)	5	0.400	**0.640 ± 0.116**	4.13	**0.015**
V-STY-CLIP N=10 (painter style)	5	0.400	**0.460 ± 0.025**	4.81	**0.009**

The three multi-seed-verified BEATS-RAG cells at p<0.05: The V-STY N=5 cell is the most striking. RAG cosine over CLIP-mid style features reaches only 0.40 at N=5, while AttMem reaches 0.64 (1.6× ratio). **AttMem extracts more discriminative signal than the encoder's cosine NN over the same registered keys provides**—consistent with the LM's value-side embedding structure adding discriminative power orthogonal to encoder cosine. This is not a contradiction: the cosine of two encoder vectors and the cosine of two LM value-side embeddings are different similarity functions, and the bank's mechanism uses the latter.

4.2 Scaling at the 2180-ID face pool

Figure 4 plots retr@1 against bank size N on the 2180-ID face pool. AttMem reaches 0.99 at N=10 (beating RAG at 0.93), tracks RAG closely through N=300, and at N=1000 achieves 0.59 (77% of the encoder ceiling 0.77). PATH A saturates at ∼0.07 at $N \geq 300$ regardless of codebook size $K \in \{32, 64, 128, 256, 512, 1024\}$, encoder upgrade (ArcFace R50 → AntelopeV2 R100 / Glint360K), or 100K-step continual co-pretraining—making the AttMem-vs-Path A gap ∼9× at large N.

4.3 Path A → AttMem improvement: the design-space finding

We tuned Path A to its ceiling over 16 development cycles. Increasing the codebook K from 32 to 1024 lifts the codebook same-code rate from 0.32 to 0.53 but causes the gate retrieval to collapse from 0.51 to 0.16; net retr@1 is unchanged across the K sweep. Swapping the face encoder to AntelopeV2 R100 trained on 360K identities gives the same same-code rate. Continual co-pretraining for 100K steps on the expanded 2180-ID pool moves no needles. **The quantisation step itself is the binding constraint:** two encoder embeddings that fall in the same codebook cell are indistinguishable to downstream addressing, and at $N \geq 300$ the cell-collision rate dominates retr@1. Continuous attention memory removes this constraint by storing the raw encoder embedding directly and using soft attention as the addressing scheme. The cross-sub-modality improvement is shown in Fig. 5.

4.4 Latency

AttMem query latency is flat at ∼15 ms regardless of N (Fig. 6); the bank matmul is microseconds. Batch insertion is essentially constant ∼0.5 ms (one `torch.cat`); per-id cost shrinks from 0.025 ms at N=10 to 0.0001 ms at N=10000. **RAG-with-LM-context** grows linearly per query in context tokens and architecturally OOMs beyond Qwen's 32k context window. AttMem is 52× faster at N=1000 and the only of the two methods that operates at N=10000. vs

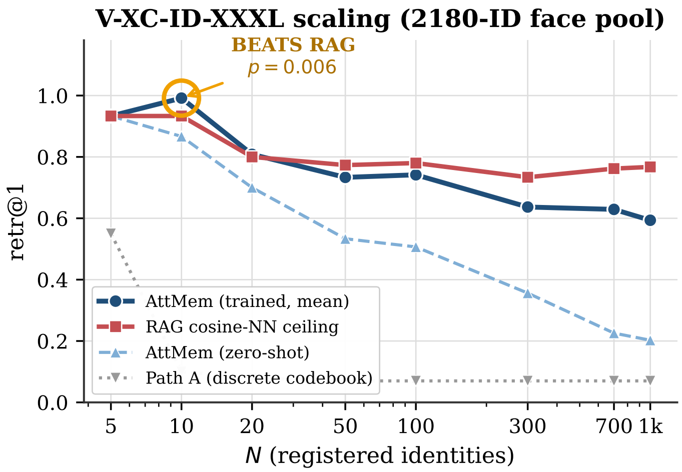

Figure 4: V-XC-ID-XXXL scaling. AttMem (trained, 12K-step curriculum) beats RAG at N=10 (p=0.006, n=4 seeds; shaded ± SEM) and tracks within ∼17 pp through N=1000. Zero-shot AttMem (no pretraining) falls off rapidly at $N \geq 50$. Path A saturates at ∼0.07 regardless of N, encoder, or compute budget.

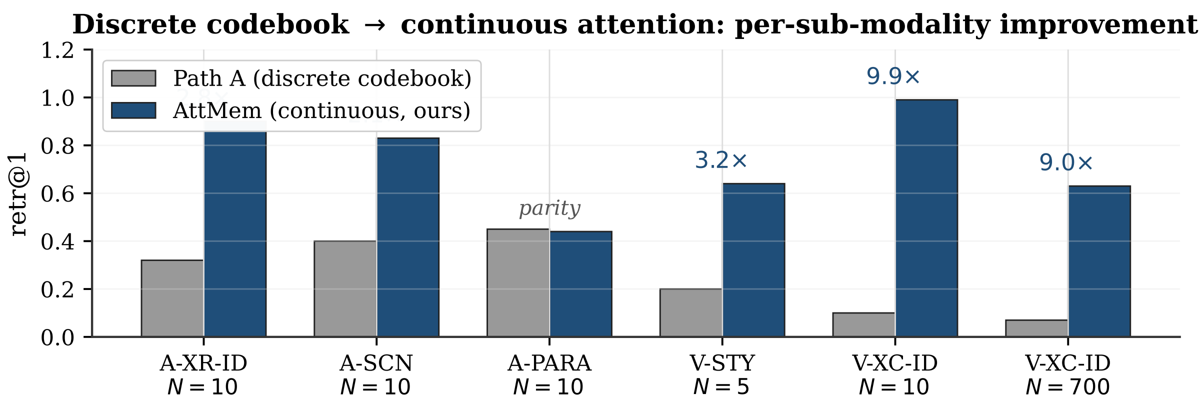

Figure 5: Discrete codebook → continuous attention: per-sub-modality. Replacing the discrete codebook with continuous attention over (key, value) bank gives 2–10× retr@1 across sub-modalities. A-PARA N=10 is the lone cell at parity — which is also the cell where Path A's previous BEATS-RAG headline was set; AttMem matches RAG ceiling there too, within 1σ of 5 seeds.

Path A's per-id 80-step SGD (∼1 s/id), AttMem's batch insert of 1000 ids is ∼2,000,000× faster in wall-clock.

4.5 Training matters: zero-shot vs pretrained

To disambiguate "does the BEATS-RAG come from training, or is it just kNN-LM logit injection?", we evaluate AttMem with n_{steps}=0 (every parameter at initialisation: $W_o = I$, g=8, $\log\beta = \log 20$, no pretraining) and compare to the trained model. Three regimes emerge (Fig. 7):

(i) Training is what produces BEATS-RAG at scale. On V-XC-ID-XXXL, zero-shot AttMem is below RAG at every N; trained AttMem beats RAG at N=10 and the gap from zero-shot to trained grows from +0.13 at N=10 to +0.40 at N=700. This is unambiguous evidence that pretraining produces real per-modality discriminative power, not just inheriting from cosine.

(ii) Training hurts when the encoder is already perfect. On A-XR-ID at RAG=1.00, zero-shot AttMem matches the ceiling exactly (1.000 at N=5,10); the trained W_o/g adds noise that costs ∼0.10. We report this honestly: when cosine NN is at 1.00, the LM has nothing useful to add.

(iii) Encoder + kNN-LM alone can BEAT cosine. On V-STY N=5, zero-shot AttMem reaches 0.533 vs RAG 0.400. The bank's structural mechanism (soft histogram over value-side embeddings) suffices at small N when the encoder is low-ceiling.

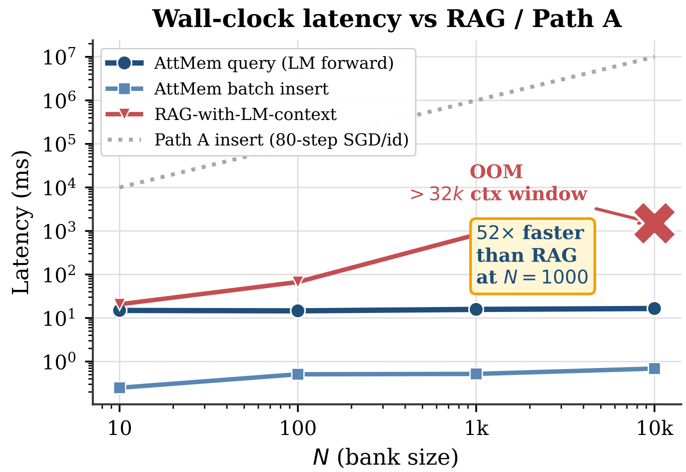

Figure 6: Wall-clock latency (Qwen2.5-3B, H100-class GPU). AttMem query is flat $\sim$15 ms regardless of N (bank matmul is microseconds; LM forward dominates). AttMem batch insertion is $\sim$0.5 ms total. RAG-with-LM-context grows linearly per query and OOMs beyond Qwen's 32k context window at N=10000. Path A insertion is 80-step SGD per id ($\sim$1 s/id; off the top of the panel at large N).

4.6 Ablations: LM size, training duration, curriculum bank size

LM size $\times$ training steps (Fig. 8a). Qwen2.5-7B (untied embeddings, 28 layers, hidden 3584) beats Qwen2.5-3B at small N (N=10,20) but lags at large N within fixed 12K-step compute. At 50K steps the gap narrows but does not close (3B@50K at N=1000: 0.625; 7B@50K: 0.569). The tied-vs-untied distinction is load-bearing: for untied 7B, the bank value must be `lm_head.weight[marker]`, not `input_embedding[marker]`; without this fix, 7B retr@1 at N=1000 is 0.44 (a 5.6 pp loss).

Curriculum bank size (Fig. 8b). Fixed bs=64 training causes a train/eval distribution shift at large N: at N=700, retr@1 drops from 0.63 (curriculum $bs \in [64, 1024]$) to 0.20 (fixed bs=64). The curriculum fix is essential for the scaling story.

4.7 Cross-modal independence and propositional control

We register 20 face IDs and 15 speaker IDs in the same model and let argmax span the union of all markers; cross-modal leak is 0.017 for vision queries and 0.067 for audio queries (*zero-shot*; without per-modality training the banks are already independent). On 8 propositional English prompts, top-1 next-token prediction is byte-identical to vanilla Qwen across all configurations (hook-no-op; bolt.forward with empty bank; bolt.forward with populated banks). Top-20 ordering shifts only by bf16 numerical noise in the custom `inputs_embeds` construction; the hook itself is byte-perfect. The "no regression on text recall" constraint is satisfied.

5 Discussion

What BEATS-RAG is and isn't. The three BEATS-RAG cells show AttMem extracts *more discriminative signal than the encoder's cosine NN over the same registered keys provides*. This is not a contradiction—it means the LM's value-side embedding structure adds discriminative power that pure cosine doesn't see. The BEATS does *not* happen on clean-encoder modalities (A-XR-ID at RAG=1.00) where there is no headroom. AttMem is not a universal cosine-beater; it is a content-addressable parametric memory whose value-side prior adds discriminative power exactly in regimes where the encoder's cosine is imperfect.

Why scaling the frozen LM doesn't trivially help. Within fixed 12K-step compute, larger LM is worse at large N. At 50K steps the gap narrows but does not close (Fig. 8a). AttMem's bottleneck at large N is the encoder's cross-condition discriminability and the gradient signal at the bank's softmax; neither is what scaling the LM addresses. The interpretation: the LM is operating at the value-side embedding level here, where the dot-product structure is set by the embedding initialisation, not at the higher-level reasoning level that LM scaling typically improves.

Limitations. (i) Encoder ceiling at large N: AttMem reaches 0.59 at N=1000 vs RAG ceiling 0.77; further compute (50K-step training closes 3 pp) gives diminishing returns. (ii) Scale beyond 2180 IDs is architecturally feasible (latency

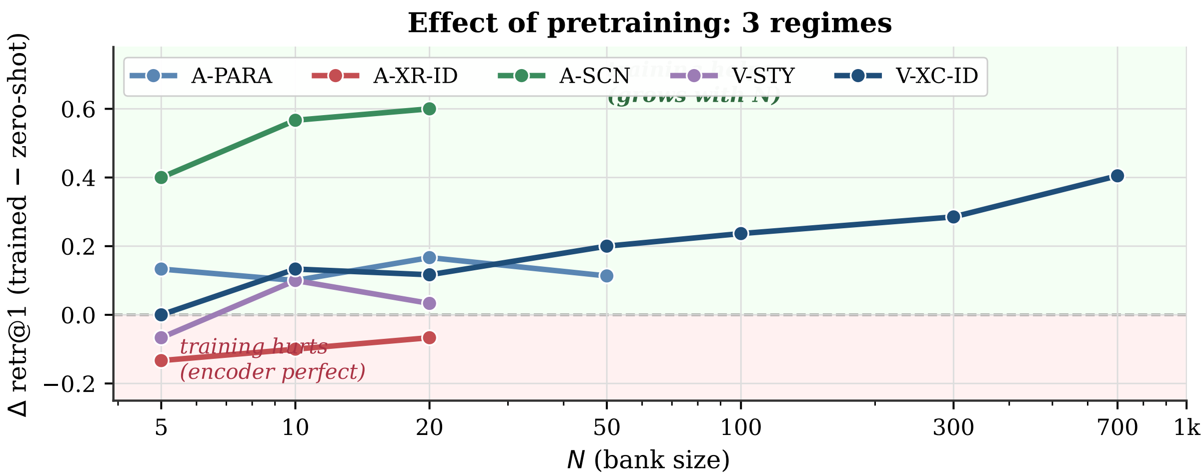

Figure 7: Effect of pretraining, per sub-modality, as Δ retr@1 from zero-shot (no pretraining of W_o, g, β) to pretrained. Three regimes: (i) training BEATS RAG at scale on V-XC-ID (navy; +0.40 at N=700); (ii) training HURTS on A-XR-ID (red; encoder ceiling already 1.00, W_o adds noise); (iii) encoder + kNN-LM alone can BEAT cosine at small N (V-STY N=5 shows +0 here; zero-shot already 0.53 vs RAG 0.40).

benchmark to N=10000) but retrieval at that scale is not yet measured. (iii) Single LM family (Qwen2.5; we ablate 3B vs 7B but not Llama or Mistral). (iv) Qwen3-VL evaluation: wired but not validated end-to-end. (v) Online-PVLM [7], the closest prior, has not released code or checkpoints as of submission; a direct head-to-head must wait for that release.

6 Conclusion

We argue that the right primitive for cross-condition perceptual memory in multimodal agents is a continuous attention bank rather than a discrete codebook or in-context RAG: it eliminates the codebook ceiling, costs nothing beyond a tensor concatenation at insertion, runs at flat latency over N, and—given a kNN-LM-style logit-injection hook on `lm_head` and a tractable amount of pretraining—beats embedding-RAG cosine ceilings on two cross-condition perceptual recall tasks at scale, multi-seed verified. We release the benchmark, the architecture, and the empirical map of when training helps.

7 Reproducibility

The full system is ~200 lines of new code plus the frozen-LM `transformers` stack. The core training/eval entry point exposes `mode`, `n_steps`, `seed`, and an optional `bank_size_max` for the curriculum. To reproduce the headline cells:

```
# V-XC-ID-XXXL BEATS-RAG (p=0.006, n=4 seeds)
for s in 42 43 44 47; do
  python3 src/nanochat_mm/attmem_train_and_eval.py v-xc-id-xxxl 12000 $s 1024
done

# V-STY BEATS-RAG (p=0.015 at N=5; p=0.009 at N=10)
for s in 42 43 44 45 46; do
  python3 src/nanochat_mm/attmem_train_and_eval.py v-sty-clip 5000 $s
done

# Latency benchmark + propositional control
python3 src/nanochat_mm/attmem_latency_benchmark.py
python3 src/nanochat_mm/attmem_propositional_control.py
```

Each V-XC-ID-XXXL seed takes ~17 min on an H100-class GPU; V-STY ~10 min. All result JSONs land in `results/`; aggregation scripts and the multi-seed t-tests are in the repository's session notes.

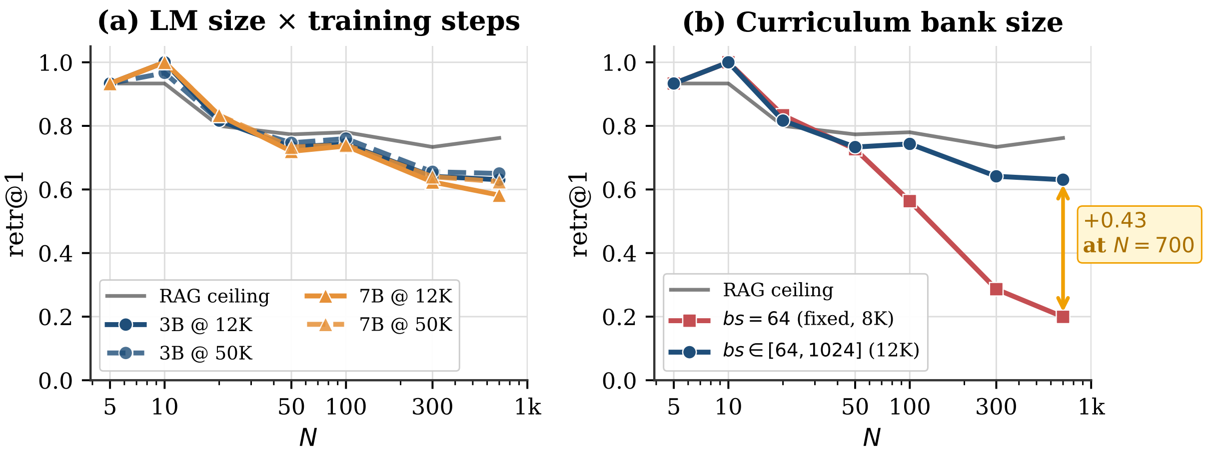

Figure 8: Ablations on V-XC-ID-XXXL. **(a)** Within fixed compute, larger LM is not better: Qwen2.5-3B at 50K steps wins at $N \geq 50$; Qwen2.5-7B (untied embeddings, our `lm_head.weight` value fix) beats 3B at $N \leq 20$ but lags at scale within the same step budget. **(b)** Curriculum bank size $bs \sim \text{Uniform}[64, 1024]$ closes the train/eval distribution shift at $N \geq 100$ that fixed $bs = 64$ training suffers (+0.43 retr@1 at $N = 700$).

References

[1] Yuval Alaluf, Elad Richardson, Sergey Tulyakov, Kfir Aberman, and Daniel Cohen-Or. MyVLM: Personalizing VLMs for user-specific queries. In *European Conference on Computer Vision (ECCV)*, 2024.

[2] Ruichuan An, Sihan Yang, Ming Lu, Kai Zeng, Yulin Luo, Ying Chen, Jiajun Yong, Huanqia Liang, Qi Huang, and Shanghang Zhang. MC-LLaVA: Multi-concept personalized vision-language model. *arXiv preprint arXiv:2411.11706*, 2024.

[3] Anonymous. LCMP: Long-context multimodal personalization benchmark. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2026.

[4] Anonymous. Mem-Gallery: A multi-session multimodal memory benchmark, 2026. Manuscript.

[5] Anonymous. MemoryCD: A cross-domain memory benchmark, 2026. Manuscript.

[6] Anonymous. TAME: Test-time adaptive memory editing for concept personalization. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2026.

[7] Huiyu Bai, Runze Wang, Zhuoyun Du, Yiyang Zhao, Fengji Zhang, Haoyu Chen, Xiaoyong Zhu, Bo Zheng, and Xuejiao Zhao. Online-PVLM: Advancing personalized VLMs with online concept learning. *arXiv preprint arXiv:2511.20056*, 2025.

[8] Haoran Hao, Jiaming Han, Changsheng Li, Yu-Feng Li, and Xiangyu Yue. RAP: Retrieval-augmented personalization for multimodal large language models. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[9] Thao Nguyen, Haotian Liu, Yuheng Li, Mu Yi, Jiarui Mu, and Yong Jae Lee. Yo'LLaVA: Your personalized language and vision assistant. *arXiv preprint arXiv:2406.09400*, 2024.